TauREx tutorial

by Maël Voyer

This tutorials aims to demonstrate how to run simple retrievals with *TauREx.* It does not covers the installation of *TauREx* or *MultiNest*. It certainly does not replace reading the documentation.

1 - What do you need?

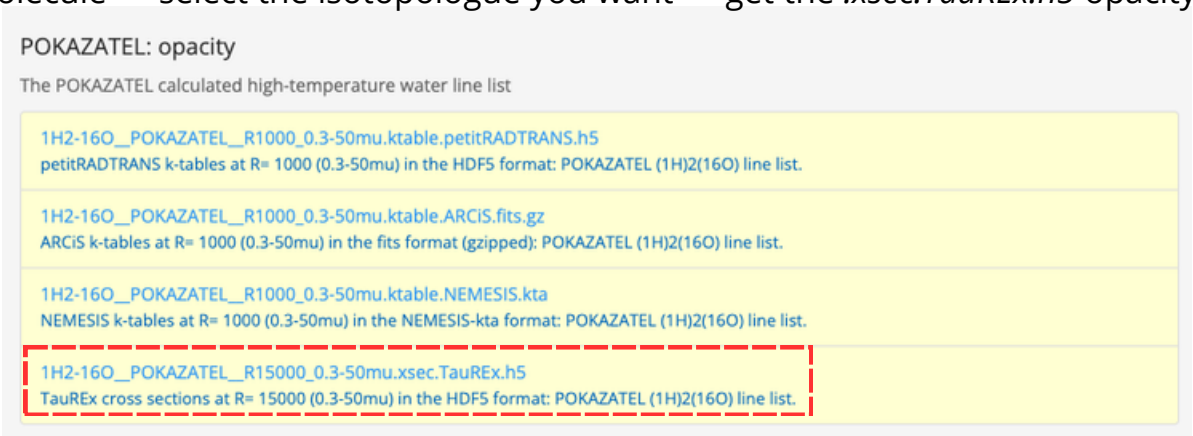
To run atmospheric retrievals the *TauREx* (Tau Retrieval for Exoplanets) code requires 3 things:

- A parameter file or **.par file**. This is the **control center** where you manage the universe.
- An observation file or **.dat file**, where the data is stored. The format is the following:

Wavelength in microns	Transit depth (Rp/Rs)2	or	Flux in W/m2/microns	Errors in the same unit		(Optional) Wavelength bin width in microns
1.1	0.0013		1.83e-17	2.3e-5	2.3e-18	0.1
1.2	0.0053		2.203e-17	1.6e-4	1.6e-19	0.1
1.3	0.00233		1.69e-18	7.4e-4	7.4e-18	0.1
1.4	0.00173		1.53e-17	8.6e-5	8.6e-19	0.1
1.5	0.00843		2.01e-19	1.9e-3	1.9e-20	0.1
...	...		...	...	...	...

- Molecular absorption data, clouds opacities, CIA opacities. Depending on what physical processes you want in the **.par** file you need different file that contains their characteristics. For example, each molecular absorbant (for example from water, CO2, etc.) needs its specific cross section (or xsec) file.

a) Molecular cross sections: the main source of cross section is EXOMOL. The path is: click on a molecule → select the isotopologue you want → get the *.xsec.TauREx.h5* opacity file.



b) CIA (Collision Induced Absorption): The data commonly use is from HITRAN.

c) Clouds absorptions: To include the absorption from condensed species (water ice, silicates, etc) either use the *TauREx-PCQ* plugin with pre-computed grids (faster) from my Zenodo or the *TauREx-PyMieScatt* plugin with optical constant from Kitzman & Heng (2018) or the table B1 in Molliere et al. (2025).

... and that's about it.

Before going on make sure you have **python env with TauREx** install both on your **local** device and on an **HPC** (equipped with MultiNest). You also need **wget**, the TauREx plugins **taurex-instrumentsystematics** and **taurex-PCQ** as well as the usual python libraries (numpy, h5py, matplotlib). Please also read the **README.txt** file provided with this tutorial.

2 - Parameter file overview

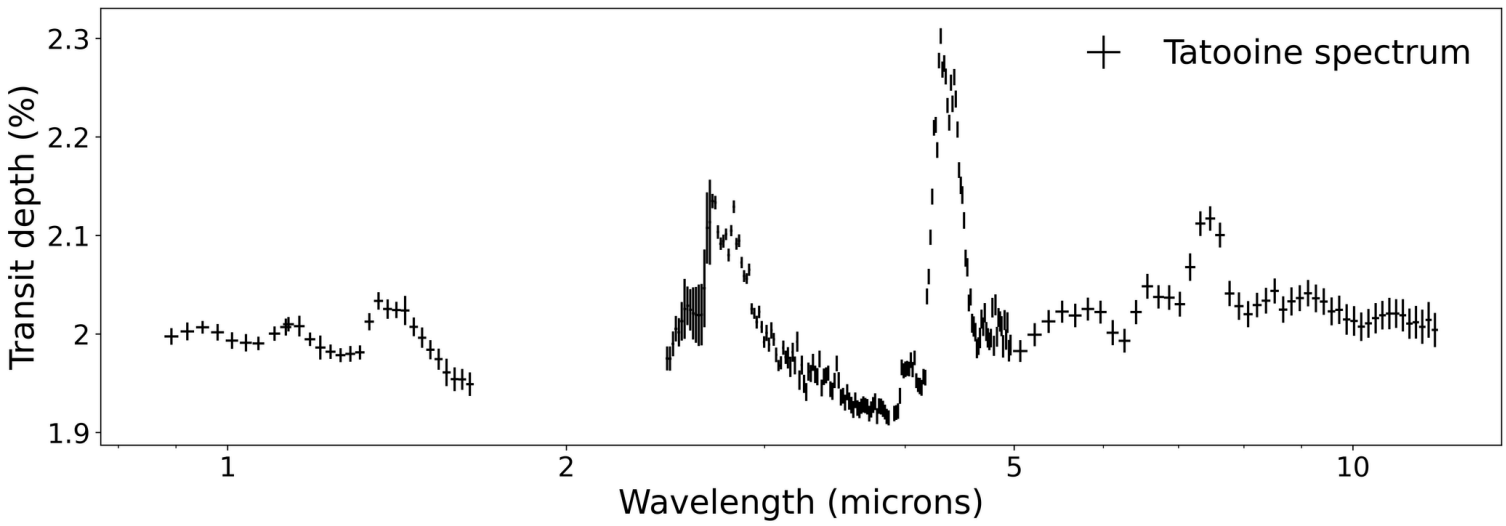
The parameter file is a text file for which we use the **.par** extension. It is organised in different **header, sub-header** and **parameters.**

```
[Header]
parameter_1 = something
parameter_2 = H2, He
parameter_3 = 1e-7

    [[Sub Header1]]
    parameter_4 = path/to/something

[Header_2]
parameter_5 = "Gaussian(mean=10 , std = 1)"
parameter_6 = True
```

The parameter file is a text file for which we use the **.par** extension. It is organised in different **header, sub-header** and **parameters.** To demonstrate how to use such files are built and used we are going to build one, brick by brick to fit the following transmission spectrum of a fictitious planet: Tatooine.



The molecular cross sections, cloud opacities, CIA data as well as the spectrum and some accompanying notebooks can be downloaded from my GitHub. Make sure you have everything downloaded and we can strat building this parameter file.

The order of the headers does not matter but parameter files can become quite dense so please somewhat follow the order suggested bellow. Now, you can create an empty text file with the .par suffix and name it was you please. While reading the following sections fill the .par file with the relevant informations.

3 - The [Global] header

This first header, [Global] fixes the paths towards the molecular cross sections (xsec) and CIA data:

```
[Global]
xsec_path = /path/to/cross_section/FOLDER
cia_path = /path/to/cia/data/FOLDER
mpi_use_shared = True
```

Take care to put the correct paths otherwise *TauREx* will not be able to function. The *mpi_use_shared* parameter controls whether MPI (the procedures that handle multi processing) can use RAM for intra node communications. This parameters improves performances when running retrievals on HPC infrastructure.

4 - The [Observation] header

The second header, [Observation] is quite simple in the default *TauREx*. Simply use the *observed_spectrum* parameter and give it the path to the Tatooine spectrum you downloaded. Use the same syntax as in the [Global] header above.
However, we will see later on that this header can quickly lengthen for complex, cross telescope and cross instrument retrievals.

5 - The [Star] header

This header controls the stellar parameters as well as the *distance* of the system in pc. For DirectImaging models this header is bypassed expect for the *distance* parameter that is used in the flux dilution factor: $F_{obs} = (R_{pl}^2 / d^2) * F_{pl}$.

The first parameter is *star_type* which controls the type of stellar model that will be used. The simplest model being a *blackbody.* In this case the stellar spectrum only depends on the star's surface *temperature* and *radius*.

```
[Star]
star_type = blackbody
temperature = 4430
radius = 0.68
distance = 30
```

For our Tatooine retrieval, use this model and the parameters shown in the adjacent image.

However, more complex stellar models are available in *TauREx* such as PHOENIX (*star_type* = *phoenix*). It's a library of synthetic stellar models that takes into account the star's *temperature*, *mass*, *radius*, *metallicity* and K band magnitude (*magnitudeK*). Note that I always refer to the taurex name of the different parameters in italic. To use the PHOENIX models you also need to give the path to the phoenix library files (*phoenix_path*).

In either model, the units of *temperature* are of course in Kelvin, the *radius*, *mass* and *metallicity* in solar units.

6 - The [Planet] header

As you might have guessed its here that we manage the bulk properties of our planet. The bulk properties are the *planet_mass* and *planet_radius* both in Jupiter units. The planet header also takes the orbital parameters, the *planet_distance* (in AU), the *impact_param*, the *orbital_period* (in days), the *albedo* and the *transit_time* (in seconds).

The Simple *planet_type* is the only implemented natively in *TauREx*. For simple transit spectroscopy like our Tatooine, only the parameters shown on the right are needed. Please use the values shown on the right image.

```
[Planet]
planet_type = Simple
planet_mass = 0.12
planet_radius = 0.85
planet_sma = 0.055
```

7 - The [Pressure] header

Now that the global headers are set we can focus on parametrising the atmospheric model! The first step is to define the boundaries of the atmosphere. We nee to set the bottom and the top of the pressure domain that the *TauREx* forward model will model.

As for the planet header, one *profile_type* is implemented: *simple* which assumes hydrostatic equilibrium. The *atm_max_pressure* and *atm_min_pressure* parameters controls the pressure boundaries. For transit spectroscopy the pressured probed are often lower (i.e., higher in the atmosphere) than in direct imaging.

```
[Pressure]
profile_type = Simple
atm_min_pressure = 1e-2
atm_max_pressure = 1e6
nlayers = 100
```

As a rule of thumb, consider using a 10 - 1 bar upper limit and 1e-2 - 1e-3 Pa lower limit for transits. In the case of direct imaging, depending on the wavelength coverage consider a 100 bar upper limit and 1e-1 Pa lower limit. That is for your first run, you may need to adjust the pressure grid later on depending on the region constrained by the retrievals.

The last parameter is *nlayers* which defines in how many layers your pressure domain will be divided. Again, there is no strict rule but use at least 80 layers and increase them if you extend the pressure domain.

As before, for the Tatooine retrieval, use the values shown in the image.

8 - The [Temperature] header

Ahah, now we are diving into the real deal! People have written entire papers on temperature profiles (Rogers 2000, Madhusudhan&Seager 2009, Guillot+2010, Gebhard 2024, Schleich+2024). Basically, the complexity of the temperature profile you use should reflect the information content present in your spectra. Indeed, if you retrieve on a HST spectra with 30 points, covering 1.1 to 1.6 microns with a signal-to-noise (S/N) ratio of 5 (i.e., a spectra with low information content) you probably can use an *isothermal* profile (i.e., constant temperature)

Multiple temperature profile are implemented in *TauREx* (full list in the documentation). Here I will focus on two of them that are, in my opinion, the most relevant: the *isothermal* profile and the *npoint*. The isothermal is quite simple, the temperature is constant along the pressure domain. Hence it only has one parameter *T*.

The *npoint* profile allows you to define as many temperature point as you want. Each temperature point is bound to a specific pressure. For example, for Tatooine we are going to use 3 temperature points:

```
[Temperature]
profile_type = npoint
T_surface= 900
T_top= 700
P_surface= 1e6
P_top= 1e-2
temperature_points=700 ,
pressure_points= 1e3 ,
```

The first temperature point, *T_surface* is set at 900 K at a pressure, *P_surface* of 10 bar (1e6 Pa).

The last temperature point, *T_top* is set at 700 K at a pressure, *P_top* of 1e-2 Pa.

In between, I choose to include one additional point. The *temperature_points* parameter which takes a **list** i.e., do not forget the comma, has one point set at 700 K. In the code this point will be referred as *T_point1*. The *pressure_points* must be of the same size as the *temperature_points* list and fixes the pressure of the in between temperature points. Here *T_point1* is fixed at 1e3 Pa (0.01 bar).

The temperature of the pressures between the defined points is interpolated by *TauREx.* Usually, we fix *P_surface* and *P_top* to the upper and lower pressure limits defined in the [Pressure] header. This is not an obligation. However, if no temperature point covers a pressure range the temperature in this region will be isothermal and take the value of the closest temperature point. For example if you set *P_surface* at 1 bar instead of 10 bar (which is the value of *atm_max_pressure*) the region between 1 and 10 bar is not covered and will be isothermal at the temperature of *T_surface*.

As before, for the Tatooine retrieval, use the values shown in the image.

9 - The [Chemistry] header

```
[Chemistry]
chemistry_type = taurex
fill_gases = H2,He
ratio=0.17

    [[H2O]]
    gas_type = constant
    mix_ratio= 1e-2

    [[CO2]]
    gas_type = constant
    mix_ratio= 1e-2

    [[SO2]]
    gas_type = constant
    mix_ratio= 1e-2
```

Now that we defined both the temperature and pressure profiles of our atmosphere we can start filling it up with molecules. Here again multiple choices for the *chemistry_type*, you can either use equilibrium chemistry, disequilibrium chemistry or *free* (also called *taurex*) chemistry.

Using either equilibrium or disequilibrium employs physically motivated chemical models. Hence, they both relies on assumption on the chemical reaction rate and their dependences on parameters like partial pressure or the temperature. To implements these models TauREx calls externals code to do the chemical computation for each retrieved model which slows down retrievals.

To start, let us ignore them and use the *free* chemistry. In this model the molecules are completely independent and can take any values within the prior range.

So, after using either *taurex* or *free* for the *chemistry_type* parameter we need to select our filling gases. For us, Tatooine will be a gaseous planet, thus dominated by di-hydrogen (*H2*) and Helium (*He*). If we wanted to respect the Star Wars lore, we could have chosen an and Earth like atmosphere and our filling gas would have been molecular nitrogen *N2*.
The ratio parameter control, well, the *ratio* between the *fill_gases*, here H2/He. We chose a solar like value of 0.17.

Then, using [[subheaders]], it's time to add molecules!

First of all, if you want to include a molecule, you need to have its cross sections in the *xsec_path* you put in the [Global] header. Then, use [[*molecule_name*]] with a tabulation beneath the [Chemistry] header as shown in the figure above. To know the correct *molecule_name* to use open the cross section file in python using the h5py library:

```
import h5py
xsec_water = h5py.File("/Users/mv277622/Desktop/TauRex_dat/xsec_50k/xsec_15k/1H2-16O__POKAZATEL__R15000_0.3-50mu.xsec.TauREx.h5", "r")
print(xsec_water['mol_name'][()][0].decode('utf-8'))
```

✓ 0.0s Python

```
H2O
```

Here for water *molecule_name* is H2O. Usually the names are what you would expect but sometimes you may be surprised. Its good to know where to check. For instance, some molecular isotopologues share the same molecule name. Then *TauREx* will load one of the two but who knows which...

After you chose a molecule, you need to select the type of profile (*gas_type* parameter) that *TauREx* will use. Two profiles are implemented: *constant* and *twolayer*. The *constant* profile imposes that the mixing ratio of the molecule be the same for every pressure. It has only one parameter: *mix_ratio* which sets the value of the molecule's mixing ratio.

The *twolayer* profile has two mixing ratio values: one at the bottom of the atmosphere *mix_ratio_surface* (i.e., higher pressure, set by *atm_max_pressure*) and another *mix_ratio_top* at the top of the atmosphere (i.e., lower pressures, set by *atm_min_pressure*). Finally, the profile needs a pressure boundary, set by *mix_ratio_P* (in Pa), that defines where the change between the *mix_ratio_surface* and the *mix_ratio_top* occurs.

For Tatooine, let us start by using 3 molecules (H2O, SO2 and CO2) with *constant* profiles. Please use the value shown in the figure.

10 - The [Model] header

Okay, up to now we gave *TauREx* a lot of informations and data but we did not actually told it how to use them. First we need to select the model geometry with the *model_type* parameter, either *transit*, *eclipse* or *directimage*.

After selecting the *model_type* we need to tell *TauREx* which physical parameters to model in the atmospheres. Specifically, which mechanisms will contribute to the optical depth (i.e., will impact the object's spectrum). These are called **contributions**. For instance, we included the [Chemistry] header but if we dot not tell TauREx to consider molecular absorption in the model, the [Chemistry] header will have no effect.

To include molecular absorption just add the [[Absorption]] sub header in the [model] section shown below.

```
[Model]
model_type = transit

    [[Absorption]]

    [[CIA]]
    cia_pairs = H2-He,H2-H2

    [[Rayleigh]]

    [[FlatMie]]
    flat_mix_ratio=5
    flat_bottomP=1e6
    flat_topP=3e3

    [[PyMieScattGridExtinction]]
    mie_particle_radius_distribution = budaj
    species =  MgSiO3 ,
    mie_particle_mean_radius = 0.1 ,
    mie_midP = 1e-2,
    mie_rangeP = 0.1 ,
    mie_particle_mix_ratio = 1e5 ,
    mie_species_path = /path/to/cloud/grid/file/MgSiO3_amorph_sol-gel_Qext_grid.h5 ,
    mie_particle_radius_Nsampling = 5
    mie_particle_radius_Dsampling = 2
    mie_particle_altitude_distrib = linear
```

Other sources of opacities often required are the [[CIA]], [[Rayleigh]] scattering, Mie scattering (i.e., by small particles, [[LeeMie]], [[BHMie]]), grey clouds (i.e., a cloud whose opacity is wavelength independent thus, often called grey or flat.). I detail below some of them.

The CIA requires data files whose path you should have set in [global]. You also need to input a list of molecule pairs to model (*cia_pairs* parameter). For gas giants, the main source of collision induced opacities are the H2-He and H2-H2 collisions.

The Rayleigh scattering only requires the [[Rayleigh]] sub header.

To include grey clouds the [[FlatMie]] sub header requires three arguments:

- the clouds bottom pressure (*flat_bottomP*). As this cloud model is very opaque we usually set this to the bottom of the atmosphere.
- the clouds mix ratio (*flat_mix_ratio*) which scales the opacity of the cloud layer.
- the clouds top pressure (*flat_topP*) defines the pressure at which the clouds stops (i.e., must be between *flat_bottomP* and *atm_min_presure*).

Including wavelength dependant aerosols (i.e., clouds) is a somewhat more difficult matter. This capacity is not included in the main *TauREx* but in side plugins as explained in the first section. Here I am going to detail how tu use TauREx-PCQ. Using TauREx-PyMieScatt is quite similar and you should not have any issues using it after following this tutorial.

TauREx-PCQ models the absorption of different condensed moelcules such as water ice, the titan tholins or the silicates. As for molecular, gas phase, absorption you need a data file for each specie you want to inclue. These files contains specie specific, wavelength dependent extinction coefficient (Qext) and are available on my Zenodo. For Tatooine please download the *MgSiO3_amorph_sol-gel_Qext_grid.h5* file and reproduce the figure above.

To include aerosol absorption from TauREx-PCQ add the [[PyMieScattGridExtinction]] subheader. Bellow is a summary of the different available parameters :

- *species*: Your name for the species included through the *mie_species_path* parameter. This name will be used as suffixes added to the other parameters to distinguish between included species.
- *mie_species_path*: Paths to the Q_ext grids of the aerosols you want to include
- *mie_particle_radius_distribution*: "normal", "budaj", or "deirmendjian"
- *mie_particle_mean_radius*: Mean particle radius (μm) of the distribution
- *mie_particle_logstd_radius*: Log-normal std dev (for "normal" distribution)
- *mie_particle_paramA/B/C/D*: Parameters for Deirmendjian distribution
- *mie_particle_mix_ratio*: Number density (molecules/m³)
- *mie_midP*: Pressure at cloud center (Pa)
- *mie_rangeP*: Extend of the clouds in log scale around mie_midP. If mie_midP = 1e5 Pa and *mie_rangeP* = 1 then clouds extend from 1e6 to 1e4 Pa
- *mie_particle_altitude_distrib*: Currently supports 'exp_decay' or 'linear'
- m*ie_particle_altitude_decay*: if mie_particle_altitude_distrib = exp, list of decay exponent (one per species) e.g., -5.

...and with all that, you can finally run a forward model.

11 - Computing a forward model

Now that we have a working parameter file we can use it to generate a *TauREx* forward model. The simplest way is through the command:
taurex -i your_par_file_name.par -p

However, we can also create the forward model inside a notebook for more flexibility. You can follow the *DEMO_run_and_plot_forward_model.ipynb* available in the repo you found this file. Note that the model you build in te notebook is fully interactive. You can modify anything. For instance to see the affect of more water on the spectrum, change the water mixing ratio, re run the model and tadaa.

12 - Setting up a retrieval

Alright, you know how to setup up a TauREx forward model but, what about doing retrievals?? We only need to setup two more header before we are ready. First, giving *TauREx* a method to explore the parameter space and then telling it which parameters to fit.

13 - The [Optimizer] header

The optimizer, is the algorithm that will explore the parameter space and give TauREx a set of parameters value to generate a forward model with. Then this model is compared to the data and the result is send back to the optimizer to guide him through the parameter space.

Ideally, you should have access to some HPC with MultiNest installed. Make sure you added the MultiNest path to $PATH. In the parameter file after the [Optimizer] header the parameter *optimizer* lets you select which algorithm to use. Compatible with *TauREx* are, polychord, ultranest, nestle and multinest.

```
[Optimizer]
optimizer = multinest
multi_nest_path=  #path/to/where/you/want/the/multinest/output
num_live_points= 300
search_multi_modes = False
resume = True
```

The most often used is MultiNest. Its parameters are:

- *multi_nest_path*: which sets the path of the output MultiNest files. Use a dedicated folder.
- *resume*: it can be *True* or *False*. If *True* and MultiNest files exists in *multi_nest_path* it will resume the retrieval from where the previous run stopped.
- *search_multi_mode*: enable MultiNest to look for multiple solutions.
- num_live_points: defines, for each iteration, how many sets of parameters must be given to *TauREx.*

14 - The [Fitting] header

The final piece, which parameters do you want TauREx to fit? For some of them you might have a pretty good idea of their values but maybe you are completely clueless to the values of others parameters. The global syntax is simple, after the [Fitting] header:
param_name:fit = *True* or *False*
param_name:prior = "Some_prior"

```
[Fitting]
planet_radius:fit = True
planet_radius:prior = "Gaussian(mean=0.85,std=0.1)"

planet_mass:fit = False

T_surface:fit = True
T_surface:prior = "Uniform(bounds=(500, 1500))"
T_point1:fit = True
T_point1:prior = "Uniform(bounds=(500, 1500))"
T_top:fit = True
T_top:prior = "Uniform(bounds=(500, 1500))"


H2O_surface:fit = True
H2O_surface:prior = "LogUniform(bounds=(-12, -1))"
H2O_top:fit = True
H2O_top:prior = "LogUniform(bounds=(-12, -1))"

H2O_P:fit = True
H2O_P:prior = "LogUniform(bounds=(6, -2))"
```

```
CO2:fit = True
CO2:prior = "LogUniform(bounds=(-12, -1))"

SO2:fit = True
SO2:prior = "LogUniform(bounds=(-12, -1))"

flat_topP:fit = True
flat_topP:prior = "LogUniform(bounds=(6, -2))"


Rmean_MgSiO3:fit=True
Rmean_MgSiO3:prior = "LogUniform(lin_bounds=(1e-3, 10))"

X_MgSiO3:fit=True
X_MgSiO3:prior = "LogUniform(bounds=(0, 15))"

midP_MgSiO3:fit=True
midP_MgSiO3:prior = "LogUniform(bounds=(6, -2))"

rangeP_MgSiO3:fit=True
rangeP_MgSiO3:prior = "Uniform(bounds=(0.1, 3))"
```

For Tatooine, use the fitting parameters above.

The prior must be set between quote. They can be:

- "Uniform(bounds=(100, 10))"
- "LogUniform(bounds=(-12, -1))"
- "LogUniform(lin_bounds=(1e-12, 1e-1))"
- "Gaussian(mean=100,std=10)"
- "LogGaussian(mean=2,std=1)"
- "LogGaussian(lin_mean=100,std=1)"

In all the parameters I presented in this tutorials, some are fitable, some are not and other can be retrieved but are never used in practice. Here is some very general advice.

The planet mass can be retrieved but it is rather difficult to constrain. Often in transit a dynamical mass (i.e., derived from radial velocity) is available and so we often fix it. As for direct imaging it is yet unclear in which situations, if any, the mass can be robustly retrieved. The radius is more often retrieved but with a tight prior. Indeed, the white light curve provides a very good constrain on the planet's radius. Usually, using a gaussian prior centred on the planet's radius from the white light curve is the way to go.

For the atmospheric properties the main levers are the temperature, molecular content and clouds structure. For the temperature, only use isothermal profiles in very simple cases such as Hubble only spectra. Otherwise, use 3 - 4 temperature points for transit/eclipse or refer to Schleich et al. (2024) for a more detailed prescription. Use flat i.e., uniform, and rather large priors. In direct imaging, use ~one temperature point per pressure unit in log (e.g, for a pressure grid from 100 bar to 1e-5 bar use 8 temperature points).

On the chemical aspect try to identify beforehand the main molecules present in your spectrum. Is it very useful to have a general knowledge of the main absorbing bands of the molecules often encountered in exoplanet atmospheres: H2O, CO2, CO, CH4, NH3 and SO2 are a good start. You can download them onto your laptop and plot them at different temperatures and pressures to get the feel of them. I included in the repository a notebook, *DEMO_plot_cross_sections.ipynb*, that does it.

For the clouds, if you have a spectrum with wavelength shorter that ~2 microns, you probably need some scattering contribution. For very cold (300K) planets consider water ice clouds. Warmer objects around 1000K may have silicates features (8 - 11 microns). For hotter objects, consider metallic aerosols like TiO, Fe, etc. It is good practice to include a grey clouds at the bottom of the atmosphere and retrieve its top pressure.

15 - Launching a retrieval

After al this hard work you are ready to lunch your retrieval!! The command is simply:

taurex -i your_parfile.par --retrieval -o you_output_name.h5

The exact configuration to run it with MPI multi-processing will vary from cluster to cluster. The most common procedure is to use slurm with **mpirun -np nb_of_process** preceding the above command and a header looking more or less like this one:

```
#!/bin/bash
#SBATCH -N 4 # Nbr of nodes
#SBATCH -J demo # Name
#SBATCH --exclusive #Exclusive usage of the nodes
#SBATCH -n 160  #160 for run  # Nbr tasks
#SBATCH -c 1 # 1 for run # core per task
#SBATCH --mail-type=BEGIN,END,FAIL  #Send emails
#SBATCH --mail-user=mael.voyer.slurm@gmail.com# send-to address
#SBATCH --out=/path/to/logs/%j.out #log file location

module purge  #purge loaded modules to be sure

module load userspace/tr17.10   #Load the relevant modules. WILL CHANGE FROM HPC TO HPC
module load anaconda3/5.1.0
module load cmake
module load gcc/7.2.0
module load openmpi/gcc72/psm2/3.0.0
module load blas/gcc72/3.7.1
module load lapack/gcc72/3.7.1

export LD_LIBRARY_PATH=/path/to/MultiNest/lib:$LD_LIBRARY_PATH  #Add MultiNest to the LD_LIBRARY_PATH.

source activate /beegfs/data/mv277622/conda_envs/python3.6  #Activate your pyhton env with taurex and its plugins

mpirun -np 160 taurex -i /path/to/parfile.par --retrieval -o /path/to/output.h5  # Launch, here on 160 parallel process.
```

To launch your script the usual command is **sbatch script_name.sh** but again this can vary from cluster to cluster. In any case, you should have some form of logs that will show you the progress of the retrieval. You can use the **tail -f your_log_file** command to watch the progress of the retrieval.

Now its time to launch the Tatooine retrieval! It may take several hours to run.

16 - Unboxing a retrieval

A *TauREx* retrieval is completed when you see this at the end of the logs file:

```
MULTINEST: an efficient and robust Bayesian inference tool for cosmology and particle physics
Feroz, F., Hobson, M. P., Bridges, M.
MNRAS, 398, 1601-1614, Oct, 2009

taurex - INFO - TauREx PROGRAM END AT 2026-02-27 06:23:48.740061 s
```

You should have a .h5 output file at the location you specified when launching the retrieval. To unbox it, very simple, run:

taurex-plot -i your_output_file.h5 -a -o your_plot_folder/

It should create the following 7 files in the folder you specified. Look through them. What do you think of your retrieval?

take particular care to the posteriors, they show you the histograms of each retrieved parameters and their two-by-two correlations. Be on the lookout for a parameters that hit their prior range. If that is the case there is probably something to improve!

- output_fit_active_mixratio.pdf
- output_fit_inactive_mixratio.pdf
- output_posteriors.pdf
- output_spectrum_contrib_sol0.pdf
- output_spectrum.pdf
- output_tau_sol0.pdf
- output_tp_profile.pdf

...continue to TauREx_demo_2/3.pdf